1 Intro, DHTML & What is JS

Lecture plan (table of contents)

- **Intro:** what JS is + 3 ways to put it in a page (in head, in body, external .js file).
- **Language:** syntax, operators, data types, pop-up boxes (alert, confirm, prompt).
- **Control:** conditions, switch, loops, functions.
- **Browser side:** DOM, events, built-in objects, debugging.

DHTML = Dynamic HTML

- Lets a web page REACT and CHANGE when the user does something.
- **DHTML = HTML + CSS + JavaScript**

HTML -> content (headings, paragraphs, lists)
CSS -> looks (font, background, layout)
JS -> behavior (logic, events, interaction)

Remember: HTML = what is shown, CSS = how it looks, JS = how it behaves.

2 About JavaScript

- Front-end scripting language made by **Netscape** for dynamic content.
- Lightweight but limited capabilities.
- Can be used as an object-oriented language.
- **Client-side**, embedded in the HTML page, **interpreted by the browser**.
- Simple, flexible and powerful for manipulating the **DOM**.

What it is used for (advantages)

- Form validation.
- React to user actions, e.g. key presses.
- Change an image on mouse-over.
- Show / hide sections of a page.
- Load and change content dynamically.

- Complex calculations.
- Custom HTML controls (e.g. scrollable table).
- AJAX functionality.

What can JS do?

- Handle events.
- Read / write HTML elements and modify the DOM tree.
- Validate form data.
- Access / modify cookies.
- Detect the user's browser and OS.
- Handle exceptions.
- Asynchronous server calls (AJAX).

3 Putting JS in a Page

First scripts

```
<html>
<body>
  <script type="text/javascript">
    alert('Hello JavaScript!');
  </script>
</body>
</html>
```

```
// same page idea, but writes text into the page
document.write('JavaScript rulez!');
```

Where can code be placed?

- <script> tag in the **head**.
- <script> tag in the **body** - not recommended.
- **External .js file** linked in the head - HIGHLY recommended; the browser caches .js files.

Watch out: the <script src=...> tag must stay EMPTY. Code written inside it will not be executed!

```
<script src="scripts.js" type="text/javascript">
</script>
```

When is JS executed?

- During page loading, OR when the browser fires an event.
- All statements run at load time.
- Some statements only DEFINE functions, to be called later.
- Event handlers attached via tag attributes run when the event fires.

```
<img src="logo.gif" onclick="alert('clicked!')" />
```

Calling a function from an event handler

```
<head>
<script type="text/javascript">
  function test(message) {
    alert(message);
  }
</script>
</head>
<body>
  <img src="logo.gif" onclick="test('clicked!')" />
</body>
```

Using an external file

```
<script src="sample.js" type="text/javascript"></script>
<button onclick="sample()">Call it</button>

// sample.js
function sample() {
  alert('Hello from sample.js!')
}
```

Typo alert: type="text/javscript" on the slide is a typo. Write text/javascript (or leave type out).

4 Syntax, Types, Strings, Arrays

- Syntax is like **C# and Java**.
- Operators: + * = != && ++ ...
- Variables are **typeless** (type never declared).
- if / else, for / while, arrays and associative arrays, functions that can return a value.

Data types

- Numbers (integer and floating-point).
- Boolean (true / false).
- String - single OR double quotes work.
- Arrays.

```
var myName = "single or double quotes are fine";
var my_array = [1, 5.3, "aaa"];
```

String operations

+ joins strings. Careful: a string plus a number gives a STRING. Convert with parseInt().

```
string1 = "fat "; string2 = "cats";
alert(string1 + string2);      // fat cats
alert("9" + 9);                // 99
alert(parseInt("9") + 9);      // 18
```

Exam favourite: what is "9" + 9 ? Answer: "99" (joined, not added).

Array operations

```
var arr = new Array();       // new empty array
var arr = [1, 2, 3, 4, 5];   // with elements
arr.push(3);                 // append
var element = arr.pop();     // remove + get last
```

```
arr.length;              // number of elements
arr.indexOf(1);          // index of element
```

5 Pop-up Boxes & Examples

- **alert** - message + [OK] button.
- **confirm** - text + [OK] and [Cancel].
- **prompt** - text + input field with a default value.

```
alert("Some text here");
confirm("Are you sure?");
prompt("enter amount", 10);   // text, default value
```

(confirm gives true / false; prompt gives the typed text as a STRING - so convert it with parseInt or a multiply.)

Example: prompt + VAT

```
price = prompt("Enter the price", "10.00");
alert('Price + VAT = ' + price * 1.2);
```

Example: sum of two numbers (form)

```
function calcSum() {
  value1 = parseInt(document.mainForm.textBox1.value);
  value2 = parseInt(document.mainForm.textBox2.value);
  sum = value1 + value2;
  document.mainForm.textBoxSum.value = sum;
}


<form name="mainForm">
  <input type="text" name="textBox1" />
  <input type="text" name="textBox2" />
  <input type="button" value="Process" onclick="calcSum()" />
  <input type="text" name="textBoxSum" readonly="readonly" />
</form>
```

Key point: form values are strings! parseInt() first, or 2 + 3 becomes "23".

6 Conditions, Loops, Functions

Comparison symbols

>	greater than	<	less than
>=	greater or equal	<=	less or equal
==	equal	!=	not equal

```
unitPrice = 1.30;
if (quantity > 100) {
  unitPrice = 1.20;
}
```

switch (works like C#)

```
switch (variable) {
  case 1:   /* do something */ break;
  case 'a': /* something else */ break;
  case 3.14: /* another code */ break;
  default:  /* something different */
}
```

Watch out: without break the code falls through into the next case.

Loops: for, while, do...while

```
var counter;
for (counter = 0; counter < 4; counter++) {
  alert(counter);
}
while (counter < 5) {
  alert(++counter);
}
```

Functions

- Split code into parts: data comes in, is processed, result returned.
- Declaring variables inside is optional; **types are never declared**.
- A function does NOT have to return a value.
- You do NOT have to pass all its arguments.
- All passed arguments are available in the **arguments** array.

```
function average(a, b, c) {
  var total;
  total = a + b + c;
  return total / 3;
}

function sum() {
  var sum = 0;
  for (var i = 0; i < arguments.length; i++)
    sum += parseInt(arguments[i]);
  return sum;
}
alert(sum(1, 2, 4));  // 7
```

7 DOM - Document Object Model

- Every HTML element is reachable through the JS **DOM API**.
- Most DOM objects can be changed by the programmer.
- The event model lets the page react to user actions.
- **Advantages:** interactive pages + update objects WITHOUT reloading the page.

Accessing elements

```
var elem = document.getElementById("some_id");
var arr = document.getElementsByName("some_name");
var imgTags = el.getElementsByTagName("img");
```

getElementsByTagName returns an array of all descendant <img> elements of el.

Changing elements (DOM manipulation)

After accessing an element you can read and write its attributes.

```
function change(state) {
  var lampImg = document.getElementById("lamp");
  lampImg.src = "lamp_" + state + ".png";
  var statusDiv = document.getElementById("statusDiv");
  statusDiv.innerHTML = "The lamp is " + state;
}


<img src="test_on.gif" onmouseover="change('off')"
    onmouseout="change('on')" />
```

Common element properties

- *Most come from HTML attributes: id, name, href, alt, title, src ...*
- ***style** - changes the INLINE CSS only (not rules from embedded / external CSS). e.g. style.width, style.marginTop, style.backgroundImage.*
- ***className** - the class attribute.*
- ***innerHTML** - all the HTML inside the element.*
- *Read-only info: tagName, offsetWidth, offsetHeight, scrollHeight, scrollTop, nodeType.*

Moving around the DOM tree

```
element.childNodes    element.parentNode
element.firstChild    element.lastChild
element.nextSibling    element.previousSibling
```

Warning: *results can differ between browsers (e.g. whitespace counted as a node).*

8 The HTML DOM Event Model

- JS can register **event handlers**.
- The browser FIRES events and sends them to the handler function.

Two ways to attach a handler

```
<img src="test.gif" onclick="imageClicked()" />


var img = document.getElementById("myImage");
img.onclick = imageClicked;
```

The event object

- Every handler receives ONE parameter with information about the event.
- Contains: the event TYPE (click, key press), WHERE it happened (mouse coordinates), the SENDER (e.g. the clicked button).
- Also the state of [Alt], [Ctrl] and [Shift] keys.
- Some browsers do not send it - they place it in document.event; property names can be browser-specific.

Common events

```
Mouse:     onclick onmousedown onmouseup
           onmouseover onmouseout onmousemove
Keys:      onkeypress onkeydown onkeyup (input fields)
Interface: onblur onfocus onscroll
Form:      onchange (inputs) onsubmit (can cancel it)
Page:      onload onunload (body element only)
```

- **onsubmit** can cancel the submission - great for form validation.
- **onload** fires when everything on the page has loaded.

```
function greet() { alert("Loaded."); }
<body onload="greet()"></body>
```

9 Browser Objects

- **window** - top node of the DOM tree, the browser window.
- **document** - the currently loaded document.
- **screen** - the user's display properties.
- **navigator** - info about the browser (slide calls it browser).

Object hierarchy

```
window
 |-- navigator
 |-- screen
 |-- history
 |-- location
 '-- document
     '-- form -> button
```

window.open()

```
var newWindow = window.open("", "sampleWindow",
  "width=300, height=100, menubar=yes, resizable=yes");
newWindow.document.write("<html><body><h1>Sample</h1></body>");
```

navigator and screen

```
alert(window.navigator.userAgent);  // browser ID


window.moveTo(0, 0);
x = screen.availWidth;
y = screen.availHeight;
window.resizeTo(x, y);  // fill the screen
```

document and location

```
document.links[0].href = "yahoo.com";
document.write("This is some <b>bold text</b>");
document.location = "http://www.yahoo.com/";  // redirect
```

- document has built-in arrays of objects on the page (e.g. links).
- document.location reads the current URL OR redirects the browser.

Form validation

```
function checkForm() {
  var valid = true;
  if (document.mainForm.firstName.value == "") {
    alert("Please type in your first name!");
    document.getElementById("firstNameError").style.display = "inline";
    valid = false;
  }
  return valid;
}

<form name="mainForm" onsubmit="return checkForm()">
```

Remember: *onsubmit="return checkForm()" - returning false CANCELS the submit.*

10 Math, Date & Timers

Math object

```
for (i = 1; i <= 20; i++) {
  var x = Math.random();   // 0 up to (not incl.) 1
  x = 10 * x + 1;
  x = Math.floor(x);       // round down -> 1..10
  document.write("Random (" + i + "): " + x + "<br/>");
}
```

Date object

```
var now = new Date();
var result = "It is now " + now;
document.getElementById("timeField").innerText = result;
```

Timers

- **setTimeout(fn, ms)** - *runs ONCE after a delay.*

- **setInterval(fn, ms)** - runs REPEATEDLY every interval.
- clearTimeout / clearInterval stop them.

```
var timer = setTimeout('bang()', 5000);  // after 5 s
clearTimeout(timer);                     // cancel

var timer = setInterval('clock()', 1000); // every 1 s
clearInterval(timer);                    // stop
```

Example: digital clock

```
function timerFunc() {
  var now = new Date();
  var hour = now.getHours();
  var min = now.getMinutes();
  var sec = now.getSeconds();
  document.getElementById("clock").value = hour + ":" + min + ":" + sec;
}
setInterval('timerFunc()', 1000);
```

Modern tip: modern code passes a function, not a string: setInterval(timerFunc, 1000).

11 Debugging JavaScript

- Modern browsers have a **JavaScript console** that reports script errors (messages differ by browser).
- **Microsoft Script Editor:** Internet Explorer add-on, supports breakpoints and watches. The statement **debugger;** opens it.
- **Firebug:** Firefox add-on for debugging JS, CSS and HTML.

Firebug can...

- Breakpoints, watches, console editor.
- Edit CSS / HTML live, see how CSS rules apply.
- Show Ajax requests and responses.

- (Firebug itself is written mostly in JavaScript.)

console object

Exists only if a debugging tool supports it. Used to write log messages at runtime.

```
console.debug(message);  console.info(message);
console.log(message);    console.warn(message);
console.error(message);
```

Modern tip: Firebug and IE are outdated - today use the browser DevTools (F12): Console, Sources, Network.

Quick Revision Sheet

DHTML	HTML + CSS + JS
JS runs where?	client side, browser interprets
Best place for code	external .js file (cached)
<script src> body	must be empty
"9" + 9	"99" (string join)
parseInt("9") + 9	18
array add / remove last	push() / pop()
alert / confirm / prompt	message / OK+Cancel / input
get element by id	document.getElementById()
change page text	element.innerHTML
change inline CSS	element.style.xxx
form check	onsubmit="return f()"
run once / repeat	setTimeout / setInterval
random 1..10	floor(10 * random() + 1)
log to console	console.log()

Common mistakes

- Code inside a <script src> tag (it is ignored).
- Forgetting parseInt on form / prompt values.
- Missing break in switch.
- style.xxx cannot read values from external CSS.
- Forgetting return false / return in onsubmit.
- Using DOM tree properties and expecting the same result in every browser.

Possible exam questions

- Explain DHTML and what each part does.
- Three ways to add JS to a page - which is best and why?
- Difference between alert, confirm and prompt.
- Name 5 DOM events. Difference between setTimeout and setInterval.
- Write a function that validates a form field.